Sistema Distribuido para la Reserva de Boletos (SRB)

Primera entrega – Informe de diseño de la arquitectura base

Pontificia Universidad Javeriana · Introducción a Sistemas Distribuidos · 2026-30 · **Nicolas Esteban Jimenez Castiblanco** · 11 de octubre de 2026

El SRB permite consultar los eventos de un mes, reservar y modificar reservas de forma síncrona y cancelarlas de forma asíncrona. Este informe presenta los modelos del sistema, el diseño de la arquitectura base (Figura 1 del enunciado) desplegada en **dos máquinas**, por indicación del profesor, los componentes previstos para escalabilidad y tolerancia a fallos, el protocolo de pruebas y la estrategia de medición. La implementación entregada cubre las operaciones de consulta y reserva; modificación y cancelación están diseñadas, pero no se ejecutan en esta entrega.

1. Modelos del sistema

Modelo	Aplicación al SRB
Arquitectónico	Cliente-servidor en **tres capas** (consumidores, negocio, almacenamiento) con servicios en procesos independientes que solo se comunican por mensajes ZeroMQ sobre TCP. El Gestor de Transacciones actúa como intermediario (broker) entre clientes y RMC; el Servicio de Persistencia es el único dueño de los datos. Patrones: request/reply con ROUTER/DEALER para el canal síncrono y pipeline PUSH/PULL para el asíncrono.
De interacción	Sistema **asíncrono** en el sentido de Coulouris: sin cotas de tiempo para entrega ni relojes sincronizados entre máquinas. Por eso se usan tiempos de espera (5 s) en el cliente y en las llamadas a persistencia, y la latencia se mide con el reloj local del cliente. Dos estilos de comunicación: bloqueante (consulta, reserva, modificación) y no bloqueante (cancelación: el cliente envía y sigue trabajando; el resultado llega después como notificación). Cada solicitud lleva un identificador único (`C1-000042`) que correlaciona petición y respuesta.
De fallos	Se consideran **fallos por caída** (crash) de los procesos Gestor y RMC y **omisión** de mensajes (pérdida o respuesta tardía), detectados por tiempo de espera. No se consideran fallos bizantinos. La persistencia se asume confiable en esta entrega: SQLite aplica transacciones ACID, de modo que una escritura se confirma completa o no se aplica. Esta entrega **no recupera** fallos; la recuperación se diseña en la sección 3.
De seguridad	Amenazas consideradas: acceso no autorizado a los puertos, suplantación del identificador de cliente, alteración de mensajes en tránsito, entradas malformadas y pérdida de integridad por reservas concurrentes. **Implementado:** validación de entradas en Gestor, RMC y Persistencia, consultas SQL parametrizadas y restricciones de integridad (`ocupados ≤ capacidad`, actualización condicional en transacción). **Previsto, no implementado:** autenticación y cifrado de canales (ZeroMQ CURVE) y restricción por firewall de los puertos de la Máquina 2 a la IP de la Máquina 1.

2. Diseño de la arquitectura base

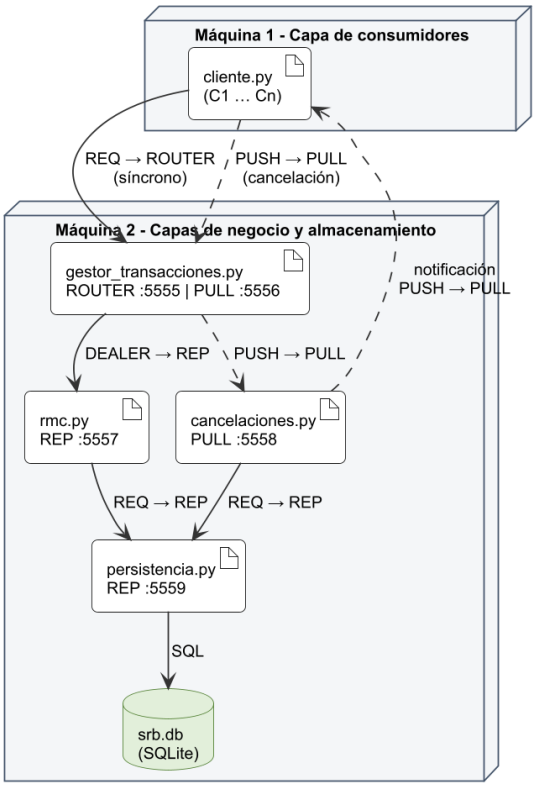


Figura 1. Diagrama de despliegue en dos máquinas.

La Máquina 1 aloja los procesos cliente y la Máquina 2 las capas de negocio y almacenamiento. Aunque Gestor, RMC y Persistencia comparten máquina, se comunican por TCP como si estuvieran separados; pasar a tres máquinas en la entrega final solo exige mover la persistencia y cambiar su dirección en `config.json`.

Enlace	Patrón	Puerto
Cliente → Gestor (consulta, reserva, modificación)	REQ → ROUTER	5555
Cliente → Gestor (cancelación)	PUSH → PULL	5556
Gestor → RMC	DEALER → REP	5557
Gestor → Cancelaciones	PUSH → PULL	5558
RMC, Cancelaciones → Persistencia	REQ → REP	5559
Cancelaciones → Cliente (notificación)	PUSH → PULL	600n (cliente)

Las solicitudes síncronas entran al Gestor por un ROUTER y salen al RMC por un DEALER que conserva la identidad del cliente, por lo que el Gestor atiende a varios clientes sin bloquearse. En la cancelación, el cliente incluye el endpoint de su socket PULL, al que Cancelaciones envía la notificación.

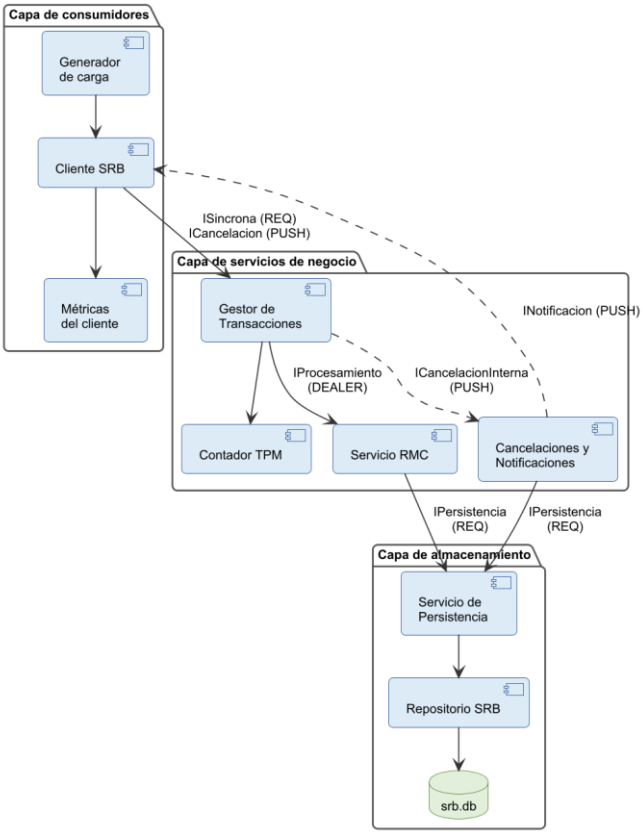


Figura 2. Diagrama de componentes.

Cada capa expone interfaces mínimas a la siguiente: el Gestor ofrece `ISincrona` e `ICancelacion` a los clientes; el RMC, `IProcesamiento`; Cancelaciones, `ICancelacionInterna`, y Persistencia, `IPersistencia`. El cliente expone `INotificacion` para recibir el resultado de una cancelación.

Modelo de datos. Tres entidades: `evento` (id, nombre, número de ocurrencias), `ocurrencia` (evento, fecha, capacidad, ocupados) y `reserva` (id, cliente, evento, fecha, asientos, estado). Se cargan 50 eventos de un mes, 20 de ellos con dos ocurrencias, con 100 asientos por ocurrencia.

Consistencia. La Persistencia es el único proceso que accede a la BD y atiende una solicitud a la vez. Cada escritura corre en una transacción que revalida el cupo antes de actualizar, así dos reservas simultáneas no pueden vender el mismo asiento.

Medición. El Gestor incluye un contador de TPM por operación y el cliente un registro de latencias, que alimentan las métricas de la sección 5.

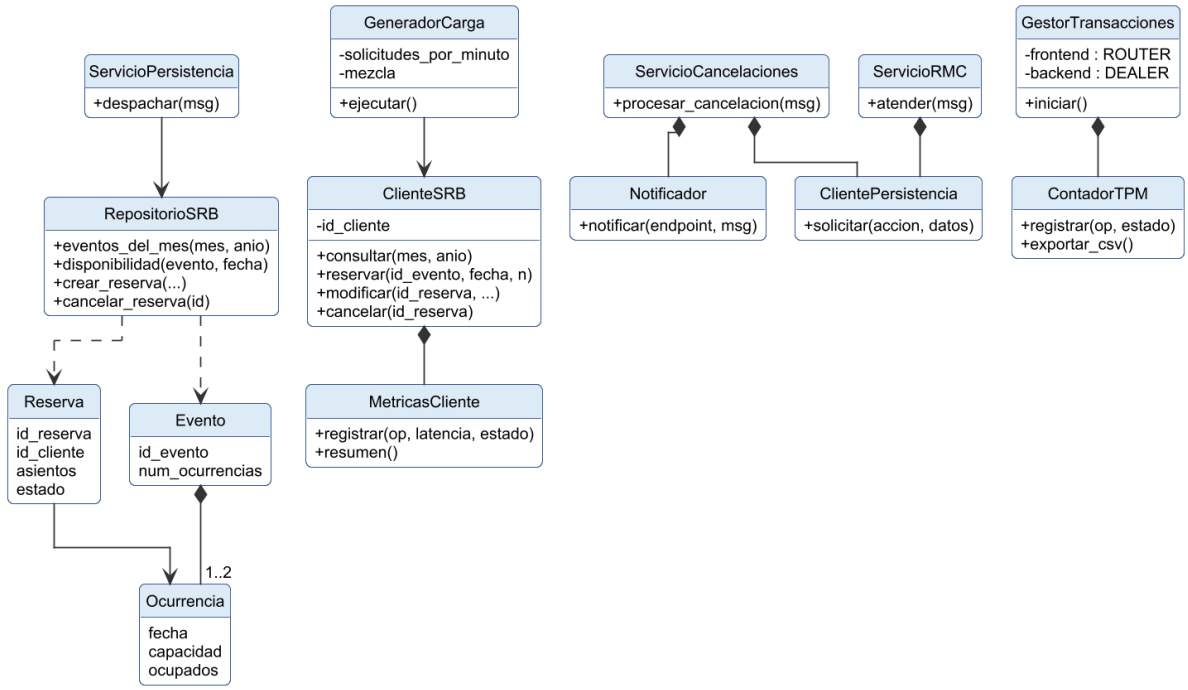


Figura 3. Diagrama de clases (clases principales de cada capa).

La reserva (Figura 4) es el flujo síncrono completo: el RMC consulta la disponibilidad y solo si hay cupo pide la confirmación; la respuesta vuelve por el mismo camino. La consulta sigue la misma ruta con la acción `EVENTOS_MES`, y la modificación agrega la lectura de la reserva actual; ambas están en `docs/diagramas`. La cancelación (Figura 5) es asíncrona: el cliente no espera y recibe la notificación después.

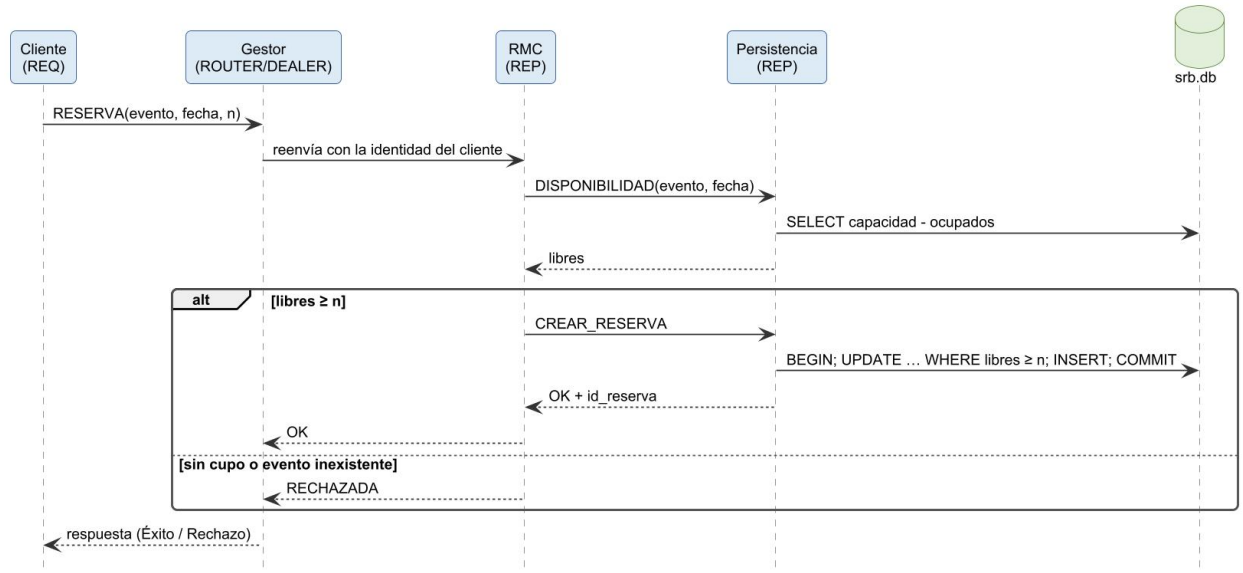


Figura 4. Diagrama de secuencia de la reserva de asientos (síncrona).

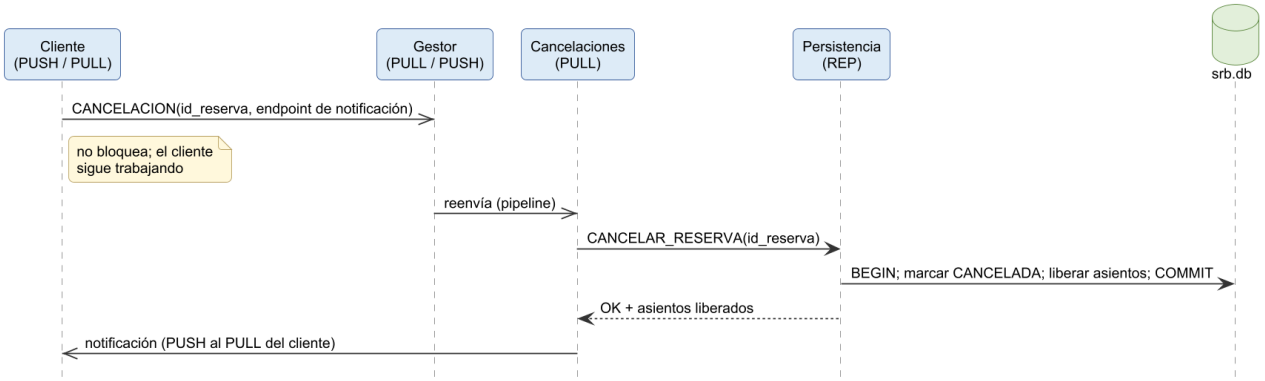


Figura 5. Diagrama de secuencia de la cancelación con notificación (asíncrona; diseñada, no ejecutada en esta entrega).

3. Componentes para escalabilidad y tolerancia a fallos

Solo se mencionan y justifican; los diagramas completos corresponden a la segunda entrega.

Componente	Atributo	Justificación
Réplicas del RMC detrás del Gestor	Escalabilidad	El RMC es el cuello de botella de las operaciones síncronas. El DEALER del Gestor reparte por turnos entre varios REP sin cambiar a clientes ni protocolo.
Réplica de la base de datos (primario/secundario)	Escalabilidad y disponibilidad	Las consultas dominan la carga: pueden leerse del secundario y las escrituras (reservas) van al primario, que además queda respaldado si falla.
Gestor de Transacciones de respaldo con latidos (heartbeat)	Tolerancia a fallos	Un respaldo en caliente detecta la caída del Gestor por ausencia de latidos y asume el puerto; los clientes reintentan hacia la misma dirección y la falla les resulta transparente.
Supervisor de procesos	Tolerancia a fallos	Detecta la caída de un RMC o del Gestor y lanza un proceso nuevo que lo sustituye, como exige el enunciado.
Identificadores de solicitud e idempotencia	Tolerancia a fallos	Con reintentos puede repetirse un mensaje; guardar los identificadores ya procesados evita reservar dos veces y permite continuar la operación donde quedó.
Cola durable de cancelaciones	Tolerancia a fallos	Una cancelación aceptada y respondida al cliente no debe perderse si cae el servicio; se guarda hasta confirmar su aplicación.

4. Protocolo de pruebas

Tipo de prueba	Qué se verifica y cómo	Criterio de aceptación
Unitarias	Repositorio (disponibilidad, creación de reserva), codificación de mensajes y análisis de parámetros del cliente, con `unittest` y BD temporal.	Resultados esperados en cada caso, incluidos los límites (0 asientos, cupo exacto).
Integración	Flujo cliente → Gestor → RMC → Persistencia con todos los servicios activos.	Cada solicitud recibe la respuesta correcta y queda reflejada en la BD.
Funcionales	Consulta, reserva exitosa y rechazada, modificación y cancelación con notificación; casos de prueba del profesor.	La BD antes y después coincide con el caso de prueba.
Concurrencia	Varios clientes reservan a la vez el mismo evento.	Nunca se supera la capacidad (sin sobreventa).
Robustez y validación	Mensajes malformados, eventos o fechas inexistentes, asientos ≤ 0.	Respuesta de error o rechazo; ningún proceso se detiene.
Desempeño y carga	3, 5 y 8 clientes durante 2–3 min en arquitectura base y escalable (Tabla 1 del enunciado).	Métricas completas y comparables entre arquitecturas.
Escalabilidad	Aumentar clientes y réplicas hasta saturar y observar el punto de quiebre.	La arquitectura escalable mantiene tiempos menores o iguales.
Tolerancia a fallos	Terminar el Gestor y el RMC en plena carga.	Un proceso nuevo asume el trabajo; sin reservas perdidas ni duplicadas.
Seguridad	Entradas con caracteres SQL, mensajes alterados y revisión de puertos expuestos.	Sin alteración de datos ni acceso desde hosts no autorizados.
Despliegue	Ejecución en dos y tres máquinas distintas.	Mismo comportamiento que en una sola máquina.

Estado actual: las pruebas de integración, funcionales y de concurrencia de consulta y reserva están automatizadas en `prueba_funcional.py` (11 verificaciones, todas superadas en una sola máquina). El resto se completa en la segunda entrega.

5. Herramientas para métricas y generación de carga

Tiempo de respuesta (promedio y desviación estándar). Cada cliente mide con `time.perf_counter()` el tiempo entre el envío y la recepción de la respuesta y guarda cada medición en `resultados/metricas_<cliente>.csv`; el resumen se calcula con el módulo `statistics` de Python (desviación estándar muestral), separado por consulta y reserva.

Transacciones por minuto (TPM). El Gestor de Transacciones cuenta las transacciones completadas por operación y por minuto de ejecución y las exporta a `resultados/tpm_gestor.csv`; el total de reservas se contrasta contra la tabla `reserva` de la BD. Los resultados se tabularán y graficarán con `pandas` y `matplotlib` para comparar la arquitectura base y la escalable en 3, 5 y 8 clientes.

Generación de carga. `cliente.py` (Python y `pyzmq`) simula un cliente por proceso con parámetros: solicitudes por minuto (mínimo 20), mezcla porcentual de operaciones, duración y semilla aleatoria; también acepta un script JSON con operaciones explícitas. Los escenarios de la Tabla 1 se obtienen lanzando varios procesos en paralelo, con `consulta=100` para los clientes que solo consultan. Se descartó JMeter y Locust como generadores primarios porque ZeroMQ no es HTTP y exigirían un cliente propio; el generador reutiliza el mismo cliente del sistema, y Locust podría envolverlo si hiciera falta.